

Vayu Air SQL Warehouse Design Challenge

Session 3 write-up: concepts used, answers, and how the code works

This document explains the warehouse design exercise in T-SQL for SQL Server. Each question connects the implementation back to the Session 3 Study Notes: OLTP versus OLAP, fact and dimension modeling, star and snowflake schemas, SCD Type 2, partitioning, medallion layers, and data contracts.

Source Context

The raw source tables are treated as bronze tables: `dbo.bronze_airports`, `dbo.bronze_aircraft`, `dbo.bronze_passengers`, `dbo.bronze_flights`, `dbo.bronze_bookings`, and `dbo.stg_passenger_updates`. The warehouse tables are built in the `dw` schema.

Revenue questions should use `booking_status = 'Confirmed'` unless the question explicitly says otherwise. For the initial fact load, however, all bookings are loaded so the fact row count matches `dbo.bronze_bookings`.

Question 1 - Grain and Candidate Columns

Study Notes Concepts Used

Concept	How it is used
Grain	Declare what one fact row means before choosing keys or measures.
Fact table	The sales fact stores the ticket sale event and additive numeric values.
Dimension table	Passenger, flight, airport, aircraft, date, fare class, and booking status describe the event.
Additivity	Measures such as fare and tax can be summed; percentages and averages cannot be blindly summed.

Answer

```
-- Grain: one row in dw.FactTicketSales represents one passenger ticket booking for one scheduled flight.
```

Source column	Classification	Warehouse interpretation
<code>booking_id</code>	Degenerate dimension	Transaction identifier kept on the fact for traceability.
<code>passenger_id</code>	Dimension key	Lookup to <code>passenger_key</code> in <code>dw.DimPassenger</code> .
<code>flight_id</code>	Dimension key	Lookup to <code>flight_key</code> in <code>dw.DimFlight</code> .
<code>booking_date</code>	Dimension key	Lookup to <code>booking_date_key</code> in <code>dw.DimDate</code> .
<code>travel_date</code>	Dimension key	Lookup to <code>travel_date_key</code> in <code>dw.DimDate</code> .
<code>fare_class</code>	Dimension attribute/key	Descriptive class such as Economy or Business.

booking_status	Dimension attribute/key	Confirmed, Cancelled, or NoShow. Revenue filters to Confirmed.
fare_amount	Measure	Additive base fare.
tax_amount	Measure	Additive tax and fee amount.
miles_earned	Measure	Usually additive loyalty miles credited.
origin_airport_code	Dimension key	Origin airport context from the joined flight.
dest_airport_code	Dimension key	Destination airport context from the joined flight.
aircraft_code	Dimension key	Aircraft context from the joined flight.
flight_date	Dimension key	Date attribute of the scheduled flight.

How It Works

The grain prevents mixed-level facts. Because one row means one ticket booking for one scheduled flight, the fact can safely store ticket_count = 1, fare_amount, tax_amount, and miles_earned. A non-additive example is load_factor_percentage or average_ticket_price: summing those values across flights would be meaningless. At a higher level they should be recalculated from base measures.

Question 2 - Star Schema DDL

Study Notes Concepts Used

Concept	How it is used
Star schema	One central fact table joins directly to dimensions.
Surrogate key	Each dimension gets a warehouse-controlled integer primary key.
Business key	Each dimension keeps the original source identifier for traceability.
OLAP design	The model is shaped for analytical reads, not transactional writes.
Conformed dimensions	DimDate, DimPassenger, DimAirport, and others can be reused by future facts.

Answer

```
CREATE SCHEMA dw;
GO

CREATE TABLE dw.DimDate (
    date_key INT NOT NULL PRIMARY KEY,
    full_date DATE NOT NULL UNIQUE,
    [year] INT NOT NULL,
    [quarter] INT NOT NULL,
    [month] INT NOT NULL,
    month_name VARCHAR(20) NOT NULL,
    [day] INT NOT NULL,
    day_name VARCHAR(20) NOT NULL
);

CREATE TABLE dw.DimPassenger (
    passenger_key INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
```

```

    passenger_id INT NOT NULL,
    passenger_name VARCHAR(200) NOT NULL,
    home_airport_code CHAR(3) NULL,
    frequent_flyer_tier VARCHAR(30) NULL,
    signup_date DATE NULL
);

CREATE TABLE dw.DimAirport (
    airport_key INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
    airport_code CHAR(3) NOT NULL UNIQUE,
    airport_name VARCHAR(200) NOT NULL,
    city VARCHAR(100) NOT NULL,
    country VARCHAR(100) NOT NULL,
    region VARCHAR(100) NOT NULL
);

CREATE TABLE dw.DimAircraft (
    aircraft_key INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
    aircraft_code VARCHAR(20) NOT NULL UNIQUE,
    model VARCHAR(100) NOT NULL,
    manufacturer VARCHAR(100) NOT NULL,
    seat_capacity INT NOT NULL
);

CREATE TABLE dw.DimFlight (
    flight_key INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
    flight_id INT NOT NULL UNIQUE,
    flight_number VARCHAR(20) NOT NULL,
    origin_airport_code CHAR(3) NOT NULL,
    dest_airport_code CHAR(3) NOT NULL,
    aircraft_code VARCHAR(20) NOT NULL,
    flight_date DATE NOT NULL
);

CREATE TABLE dw.FactTicketSales (
    ticket_sales_key BIGINT IDENTITY(1,1) NOT NULL PRIMARY KEY,
    booking_id INT NOT NULL,
    booking_date_key INT NOT NULL,
    travel_date_key INT NOT NULL,
    passenger_key INT NOT NULL,
    flight_key INT NOT NULL,
    origin_airport_key INT NOT NULL,
    dest_airport_key INT NOT NULL,
    aircraft_key INT NOT NULL,
    ticket_count INT NOT NULL DEFAULT 1,
    fare_amount DECIMAL(18,2) NOT NULL,
    tax_amount DECIMAL(18,2) NOT NULL,
    miles_earned INT NOT NULL,
    CONSTRAINT FK_FactTicketSales_Passenger
        FOREIGN KEY (passenger_key) REFERENCES dw.DimPassenger(passenger_key)
);

```

How It Works

CREATE SCHEMA dw creates a separate warehouse namespace. INT IDENTITY(1,1) tells SQL Server to auto-generate integer surrogate keys starting at 1 and increasing by 1. PRIMARY KEY enforces uniqueness in the dimension. The original business keys, such as passenger_id, airport_code, aircraft_code, and flight_id, are still kept so the warehouse row can be traced back to the source system.

The CONSTRAINT ... FOREIGN KEY clauses protect referential integrity. For example, a passenger_key in dw.FactTicketSales must already exist in dw.DimPassenger. The fact table stores only additive measures:

ticket_count, fare_amount, tax_amount, and miles_earned. Values such as aircraft model or passenger tier are descriptive attributes and belong in dimensions.

Question 3 - Load Dimensions and Fact

Study Notes Concepts Used

Concept	How it is used
Medallion flow	Raw bronze source data is transformed into curated warehouse tables.
Surrogate key assignment	Dimensions assign new keys, then the fact resolves those keys through joins.
Date dimension	A yyyymmdd integer date_key supports date joins and reporting.
Referential integrity	The load should produce no unresolved dimension references.

Answer

```
INSERT INTO dw.DimDate (date_key, full_date, [year], [quarter], [month], month_name, [day], day_name)
SELECT DISTINCT
    CONVERT(INT, CONVERT(CHAR(8), d.full_date, 112)) AS date_key,
    d.full_date,
    YEAR(d.full_date),
    DATEPART(QUARTER, d.full_date),
    MONTH(d.full_date),
    DATENAME(MONTH, d.full_date),
    DAY(d.full_date),
    DATENAME(WEEKDAY, d.full_date)
FROM (
    SELECT booking_date AS full_date FROM dbo.bronze_bookings
    UNION
    SELECT travel_date FROM dbo.bronze_bookings
    UNION
    SELECT flight_date FROM dbo.bronze_flights
    UNION
    SELECT signup_date FROM dbo.bronze_passengers
) d
WHERE d.full_date IS NOT NULL;

INSERT INTO dw.DimAirport (airport_code, airport_name, city, country, region)
SELECT airport_code, airport_name, city, country, region
FROM dbo.bronze_airports;

INSERT INTO dw.DimAircraft (aircraft_code, model, manufacturer, seat_capacity)
SELECT aircraft_code, model, manufacturer, seat_capacity
FROM dbo.bronze_aircraft;

INSERT INTO dw.DimPassenger (passenger_id, passenger_name, home_airport_code, frequent_flyer_tier,
signup_date)
SELECT passenger_id, passenger_name, home_airport_code, frequent_flyer_tier, signup_date
FROM dbo.bronze_passengers;

INSERT INTO dw.DimFlight (flight_id, flight_number, origin_airport_code, dest_airport_code,
aircraft_code, flight_date)
SELECT flight_id, flight_number, origin_airport_code, dest_airport_code, aircraft_code, flight_date
FROM dbo.bronze_flights;
INSERT INTO dw.FactTicketSales (
    booking_id, booking_date_key, travel_date_key, passenger_key, flight_key,
    origin_airport_key, dest_airport_key, aircraft_key,
    ticket_count, fare_amount, tax_amount, miles_earned
)
```

```

SELECT
    b.booking_id,
    bd.date_key,
    td.date_key,
    p.passenger_key,
    f.flight_key,
    oa.airport_key,
    da.airport_key,
    ac.aircraft_key,
    1,
    b.fare_amount,
    b.tax_amount,
    b.miles_earned
FROM dbo.bronze_bookings b
JOIN dbo.bronze_flights bf ON b.flight_id = bf.flight_id
JOIN dw.DimDate bd ON b.booking_date = bd.full_date
JOIN dw.DimDate td ON b.travel_date = td.full_date
JOIN dw.DimPassenger p ON b.passenger_id = p.passenger_id
JOIN dw.DimFlight f ON b.flight_id = f.flight_id
JOIN dw.DimAirport oa ON bf.origin_airport_code = oa.airport_code
JOIN dw.DimAirport da ON bf.dest_airport_code = da.airport_code
JOIN dw.DimAircraft ac ON bf.aircraft_code = ac.aircraft_code;

```

How It Works

The dimension loads copy distinct descriptive entities from the bronze tables. The identity columns generate surrogate keys automatically. DimDate is built from all dates needed by the model and uses style 112 in SQL Server to produce yyyyymmdd, such as 20241201.

The fact load joins bronze_bookings to each dimension using the source business keys, then inserts the surrogate keys into the fact. It should load every booking, not only Confirmed rows, because the acceptance criterion says the fact row count must match dbo.bronze_bookings. Revenue filters are applied later in analytical queries.

```

SELECT
    (SELECT COUNT(*) FROM dbo.bronze_bookings) AS bronze_booking_count,
    (SELECT COUNT(*) FROM dw.FactTicketSales) AS fact_ticket_sales_count;

SELECT COUNT(*) AS unresolved_fact_rows
FROM dw.FactTicketSales fs
LEFT JOIN dw.DimPassenger p ON fs.passenger_key = p.passenger_key
LEFT JOIN dw.DimFlight f ON fs.flight_key = f.flight_key
LEFT JOIN dw.DimAircraft ac ON fs.aircraft_key = ac.aircraft_key
WHERE p.passenger_key IS NULL
    OR f.flight_key IS NULL
    OR ac.aircraft_key IS NULL;

```

Question 4 - Snowflake the Geography

Study Notes Concepts Used

Concept	How it is used
Snowflake schema	A dimension hierarchy is normalized into smaller related tables.
Normalization	Repeated city, country, and region values are stored once.
Trade-off	Snowflaking reduces redundancy but adds joins for BI queries.

Answer

The geography in bronze_airports is denormalized. The snowflaked design is Country -> City -> Airport. Because dw.DimAirport already existed from the star schema, the practical correction is to create DimCountry and DimCity, add city_key to the existing DimAirport, and then optionally remove the repeated city, country, and region columns after validation.

```
IF OBJECT_ID('dw.DimCountry', 'U') IS NULL
BEGIN
    CREATE TABLE dw.DimCountry (
        country_key INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
        country_name VARCHAR(100) NOT NULL,
        region VARCHAR(100) NOT NULL,
        CONSTRAINT UQ_DimCountry_CountryName UNIQUE (country_name)
    );
END;
GO

INSERT INTO dw.DimCountry (country_name, region)
SELECT DISTINCT country, region
FROM dbo.bronze_airports ba
WHERE NOT EXISTS (
    SELECT 1 FROM dw.DimCountry c
    WHERE c.country_name = ba.country
);
GO

IF OBJECT_ID('dw.DimCity', 'U') IS NULL
BEGIN
    CREATE TABLE dw.DimCity (
        city_key INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
        city_name VARCHAR(100) NOT NULL,
        country_key INT NOT NULL,
        CONSTRAINT FK_DimCity_Country
            FOREIGN KEY (country_key) REFERENCES dw.DimCountry(country_key),
        CONSTRAINT UQ_DimCity_CityCountry UNIQUE (city_name, country_key)
    );
END;
GO

INSERT INTO dw.DimCity (city_name, country_key)
SELECT DISTINCT ba.city, c.country_key
FROM dbo.bronze_airports ba
JOIN dw.DimCountry c ON ba.country = c.country_name
WHERE NOT EXISTS (
    SELECT 1 FROM dw.DimCity ci
    WHERE ci.city_name = ba.city
        AND ci.country_key = c.country_key
);
GO

IF COL_LENGTH('dw.DimAirport', 'city_key') IS NULL
BEGIN
    ALTER TABLE dw.DimAirport ADD city_key INT NULL;
END;
GO

UPDATE a
SET a.city_key = ci.city_key
FROM dw.DimAirport a
JOIN dbo.bronze_airports ba ON a.airport_code = ba.airport_code
JOIN dw.DimCountry c ON ba.country = c.country_name
JOIN dw.DimCity ci ON ba.city = ci.city_name
        AND c.country_key = ci.country_key;
GO

ALTER TABLE dw.DimAirport
```

```

ADD CONSTRAINT FK_DimAirport_City
FOREIGN KEY (city_key) REFERENCES dw.DimCity(city_key);
GO

```

How It Works

DimCountry stores each country once with its region. DimCity stores each city once per country. DimAirport now points to DimCity through city_key. That makes the geography hierarchy normalized. Queries that need country or region now join Airport -> City -> Country.

The earlier error 'There is already an object named DimAirport' happened because question 2 had already created dw.DimAirport. The corrected pattern modifies the existing table instead of trying to recreate it.

Question 5 - SCD Type 2 Passenger Dimension

Study Notes Concepts Used

Concept	How it is used
Slowly Changing Dimension	Passenger attributes such as home airport and tier can change over time.
SCD Type 2	Old rows are expired and new current rows are inserted.
Effective dates	effective_from and effective_to to define the time span of each version.
is_current	Marks the active version for fast current-state lookups.
Surrogate key per version	The same passenger_id can have multiple passenger_key values across history.

Answer

If the fact table already references dw.DimPassenger, do not drop the passenger dimension. SQL Server will block the drop because of the foreign key. Instead, convert the existing dimension by adding SCD2 columns.

```

IF COL_LENGTH('dw.DimPassenger', 'effective_from') IS NULL
    ALTER TABLE dw.DimPassenger ADD effective_from DATETIME2(0) NULL;
GO

IF COL_LENGTH('dw.DimPassenger', 'effective_to') IS NULL
    ALTER TABLE dw.DimPassenger ADD effective_to DATETIME2(0) NULL;
GO

IF COL_LENGTH('dw.DimPassenger', 'is_current') IS NULL
    ALTER TABLE dw.DimPassenger ADD is_current BIT NULL;
GO

UPDATE dw.DimPassenger
SET effective_from = CAST(signup_date AS DATETIME2(0)),
    effective_to = NULL,
    is_current = 1
WHERE effective_from IS NULL OR is_current IS NULL;
GO

ALTER TABLE dw.DimPassenger ALTER COLUMN effective_from DATETIME2(0) NOT NULL;
ALTER TABLE dw.DimPassenger ALTER COLUMN is_current BIT NOT NULL;
GO

```

```

CREATE UNIQUE INDEX UX_DimPassenger_Current
ON dw.DimPassenger(passenger_id)
WHERE is_current = 1;
GO
DECLARE @load_datetime DATETIME2(0) = SYSUTCDATETIME();

DROP TABLE IF EXISTS #passenger_changes;

SELECT DISTINCT
    s.passenger_id,
    s.passenger_name,
    s.home_airport_code,
    s.frequent_flyer_tier
INTO #passenger_changes
FROM dbo.stg_passenger_updates s
JOIN dw.DimPassenger d
    ON s.passenger_id = d.passenger_id
    AND d.is_current = 1
WHERE COALESCE(s.home_airport_code, '###') <> COALESCE(d.home_airport_code, '###')
    OR COALESCE(s.frequent_flyer_tier, '###') <> COALESCE(d.frequent_flyer_tier, '###');

UPDATE d
SET d.is_current = 0,
    d.effective_to = @load_datetime
FROM dw.DimPassenger d
JOIN #passenger_changes c ON d.passenger_id = c.passenger_id
WHERE d.is_current = 1;

INSERT INTO dw.DimPassenger (
    passenger_id, passenger_name, home_airport_code, frequent_flyer_tier,
    signup_date, effective_from, effective_to, is_current
)
SELECT
    passenger_id, passenger_name, home_airport_code, frequent_flyer_tier,
    NULL, @load_datetime, NULL, 1
FROM #passenger_changes;

```

How It Works

The SCD2 process has two steps for changed passengers. First it expires the current row by setting `is_current = 0` and `effective_to` to the load timestamp. Second it inserts a new row with the same `passenger_id`, a new `passenger_key`, `effective_from` equal to the load timestamp, `effective_to = NULL`, and `is_current = 1`.

The `COALESCE` comparison avoids missing changes when one side is `NULL`. The filtered unique index enforces the rule that each `passenger_id` can have only one current version.

```

SELECT passenger_id, COUNT(*) AS current_version_count
FROM dw.DimPassenger
WHERE is_current = 1
GROUP BY passenger_id
HAVING COUNT(*) <> 1;

```

The validation query above should return no rows.

Question 6 — Partition the Sales Fact

Concepts from the Study Notes

This question applies the following concepts:

- **Partitioning:** splitting a large table into physical sections based on a key, usually a date.
- **Partition function:** defines the boundary values that determine which rows belong to each partition.
- **Partition scheme:** maps the partitions created by the partition function to SQL Server filegroups.
- **Partition pruning:** SQL Server skips partitions that cannot contain rows matching the query filter.
- **Fact table:** `dw.FactTicketSales` is the largest and fastest-growing table in the warehouse, so it is the best candidate for partitioning.
- **OLAP physical design:** partitioning improves analytical queries that scan large volumes of data, particularly when filtering by date.
- **Dominant filter:** the fact table was partitioned by `booking_date_key` because sales reports commonly filter by booking date.

Table partitioning is different from the window-function `PARTITION BY`. Table partitioning changes how rows are physically stored, while window `PARTITION BY` only groups rows for a calculation.

Create the Partition Function and Partition Scheme

The partition function defines monthly boundaries for `booking_date_key`.

```
CREATE PARTITION FUNCTION pf_FactTicketSales_Q6 (INT)
AS RANGE RIGHT FOR VALUES (
    20240101, 20240201, 20240301, 20240401,
    20240501, 20240601, 20240701, 20240801,
    20240901, 20241001, 20241101, 20241201,
    20250101
);
GO
```

```
CREATE PARTITION SCHEME ps_FactTicketSales_Q6
AS PARTITION pf_FactTicketSales_Q6
ALL TO ([PRIMARY]);
GO
```

`RANGE RIGHT` means that each boundary value belongs to the partition on its right. For example, `20241201` belongs to the partition containing December 2024.

The partition scheme stores all partitions in the `PRIMARY` filegroup. In a production warehouse, partitions could be distributed across different filegroups, but using `PRIMARY` is sufficient for this exercise.

Place the Fact Table on the Partition Scheme

The existing primary key must first be removed because the table already has a clustered index. The name of the primary-key constraint is retrieved from SQL Server metadata.

```
DECLARE @pk_name SYSNAME;
DECLARE @sql NVARCHAR(MAX);

SELECT @pk_name = kc.name
FROM sys.key_constraints kc
WHERE kc.parent_object_id = OBJECT_ID('dw.FactTicketSales')
      AND kc.type = 'PK';

IF @pk_name IS NOT NULL
BEGIN
    SET @sql =
        N'ALTER TABLE dw.FactTicketSales DROP CONSTRAINT '
        + QUOTENAME(@pk_name) + N';';

    EXEC sys.sp_executesql @sql;
END;
GO

ALTER TABLE dw.FactTicketSales
ADD CONSTRAINT PK_FactTicketSales_Q6
PRIMARY KEY CLUSTERED (
    booking_date_key,
    ticket_sales_key
)
ON ps_FactTicketSales_Q6(booking_date_key);
GO
```

QUOTENAME() safely formats the primary-key constraint name.

sp_executesql executes the dynamically created ALTER TABLE statement.

The primary key is recreated as a clustered index beginning with booking_date_key. The clause:

```
ON ps_FactTicketSales_Q6(booking_date_key)
```

places the clustered index, and therefore the fact table, on the partition scheme.

Confirm That the Fact Uses the Partition Scheme

```
SELECT
    i.name AS index_name,
    ds.name AS storage_location,
    ds.type_desc
FROM sys.indexes i
```

```

INNER JOIN sys.data_spaces ds
    ON i.data_space_id = ds.data_space_id
WHERE i.object_id = OBJECT_ID('dw.FactTicketSales')
    AND i.index_id = 1;

```

The result was:

index_name	storage_location	type_desc
PK_FactTicketSales_Q6	ps_FactTicketSales_Q6	PARTITION_SCHEME

This confirms that `dw.FactTicketSales` physically sits on the partition scheme.

Compare the Execution Plans

The actual execution plan was enabled in SSMS using:

Ctrl + M

Query 1 — Filter on the Partition Key

```

SELECT COUNT_BIG(*) AS partition_key_count
FROM dw.FactTicketSales
WHERE booking_date_key >= 20241201
    AND booking_date_key < 20250101
OPTION (RECOMPILE);

```

This query filters directly on `booking_date_key`, which is the partition key.

The actual execution plan showed approximately:

```

Actual Partition Count: 1
Actual Partitions Accessed: 13

```

The screenshot displays the SQL Server Enterprise Manager interface. The top pane shows two queries. Query 1 filters on `booking_date_key` and uses a `Clustered Index Seek (Clustered)` operator. Query 2 filters on `travel_date_key` and uses a `Clustered Index Scan (Clustered)` operator. The bottom pane shows the execution plans for both queries. The right pane shows the properties for the `Clustered Index Seek (Clustered)` operator, including actual execution statistics.

Clustered Index Seek (Clustered)	
Misc	
Actual Execution Mode	Row
Actual I/O Statistics	
Actual Number of Batches	0
Actual Number of Rows for	1864
Actual Number of Rows Rea	1864
Actual Partition Count	1
Actual Partitions Accessed	13
Actual Rebinds	0
Actual Rewinds	0
Actual Time Statistics	
Defined Values	
Description	Scanning a particular range of
Estimated CPU Cost	0,0022034
Estimated Execution Mode	Row
Estimated I/O Cost	0,0157176
Estimated Number of Execu	1
Estimated Number of Rows	1860,4
Estimated Number of Rows	1860,4
Estimated Number of Rows	1860,4
Estimated Operator Cost	0,017921 (94%)
Estimated Rebinds	0
Estimated Rewinds	0
Estimated Row Size	9 B
Estimated Subtree Cost	0,017921
Actual Execution Mode	
Actual Execution Mode	

SQL Server accessed only partition 13 because that partition contains the December 2024 booking dates.

This demonstrates **partition pruning**: SQL Server skipped the partitions that could not contain matching rows.

Query 2 — Filter on a Non-Partition Column

```
SELECT COUNT_BIG(*) AS non_partition_key_count
FROM dw.FactTicketSales
WHERE travel_date_key >= 20241201
      AND travel_date_key < 20250101
OPTION (RECOMPILE);
```

This query filters on `travel_date_key`, which is not the partitioning column.

Although `travel_date_key` is also a date, SQL Server cannot use the `booking_date_key` partition boundaries to determine which partitions contain the matching rows. The execution plan therefore accessed multiple or all partitions.

The screenshot displays the SQL Server Enterprise Manager interface. The top pane shows two SQL queries. Query 1 filters on `booking_date_key` and includes a comment indicating it should prune partitions. Query 2 filters on `travel_date_key` and includes a comment indicating it should access multiple partitions. The bottom pane shows the execution plan for Query 2, which is a `Clustered Index Scan (Clustered)` on `[FactTicketSales].[PK_FactTicketSales]`. The Properties window on the right shows actual execution statistics for this scan, including actual number of rows (40000), actual number of partitions accessed (1..14), and estimated operator cost (0.360318, 98%).

Conclusion

The partition function defines the booking-date ranges, and the partition scheme determines where those partitions are stored. Rebuilding the clustered primary key on the partition scheme physically partitions `dw.FactTicketSales` by `booking_date_key`.

The first query benefits from partition pruning because it filters directly on the partition key. The second query does not benefit because it filters on `travel_date_key`, which is unrelated to the table's physical partition boundaries.

Question 7 - Medallion Mapping and Data Contract

Study Notes Concepts Used

Concept	How it is used
Medallion architecture	Bronze is raw, silver is cleaned/conformed, and gold is curated for analytics.
Data contract	A producer-consumer agreement defines schema, allowed values, quality, freshness, and change rules.
Breaking change	Renaming, removing, or retyping fields can break downstream consumers.
Non-breaking change	Adding an optional nullable field is usually safe.

Answer Part A - Medallion Layer Mapping

Table	Layer	Reason
dbo.bronze_airports	Bronze	Raw source table; geography is still denormalized.
dbo.bronze_aircraft	Bronze	Raw aircraft reference data from the source system.
dbo.bronze_passengers	Bronze	Raw current passenger state from the source system.
dbo.bronze_flights	Bronze	Raw flight schedule data from the source system.
dbo.bronze_bookings	Bronze	Raw ticket sales feed before warehouse modeling.
dbo.stg_passenger_updates	Bronze	Raw daily passenger change feed used for SCD2.
dw.DimDate	Gold	Curated date dimension for analytical queries.
dw.DimPassenger	Gold	Curated SCD2 passenger dimension preserving history.
dw.DimFlight	Gold	Curated flight dimension for slicing ticket sales.
dw.DimAircraft	Gold	Curated aircraft dimension for aircraft analysis.
dw.DimAirport	Gold	Curated airport dimension for origin and destination analysis.
dw.DimCity	Gold	Snowflaked geography dimension.
dw.DimCountry	Gold	Snowflaked geography dimension storing country and region once.
dw.FactTicketSales	Gold	Curated, partitioned sales fact at ticket-booking grain.

In this exercise, separate physical silver tables were not built. The cleaning and conforming logic happens during the load into dw tables, and the final star/snowflake warehouse tables are the gold layer.

Answer Part B - Data Contract for dbo.bronze_bookings

Column	Type	Rule / Meaning
booking_id	INT	Unique booking identifier. Required.
passenger_id	INT	Passenger business key. Required.
flight_id	INT	Flight business key. Required.
booking_date	DATE	Date the ticket was booked. Required.
travel_date	DATE	Date the passenger travels. Required.
fare_class	VARCHAR	Allowed: Economy, Premium Economy, Business, First.
fare_amount	DECIMAL	Base fare in INR. Required and ≥ 0 .
tax_amount	DECIMAL	Taxes and fees in INR. Required and ≥ 0 .
booking_status	VARCHAR	Allowed: Confirmed, Cancelled, NoShow.

miles_earned	INT	Loyalty miles credited. Required and >= 0.
--------------	-----	--

Freshness / SLA: the feed must be delivered daily by 06:00 IST and include all bookings created or updated through the previous business day.

Owner: Vayu Air Booking Platform Team.

Revenue rule: revenue calculations must use booking_status = 'Confirmed' unless a specific task or report says otherwise.

Breaking change example: renaming fare_amount to base_fare_amount, removing booking_status, or changing booking_id from INT to VARCHAR. These can break downstream warehouse loads and reports.

Non-breaking change example: adding a new nullable optional column such as payment_method VARCHAR(50). Existing consumers can ignore it.